

3. Get the Groups

As new students begin to arrive at college, each receives a unique ID number, 1 to n . Initially, the students do not know one another, and each has a different circle of friends. As the semester progresses, other groups of friends begin to form randomly.

There will be three arrays, each aligned by an index. The first array will contain a *queryType* which will be either *Friend* or *Total*. The next two arrays, *students1* and *students2*, will each contain a student ID. If the query type is *Friend*, the two students become friends. If the query type is *Total*, report the sum of the sizes of each group of friends for the two students.

Example

$n = 4$

queryType = ['Friend', 'Friend', 'Total']

student1 = [1, 2, 1]

student2 = [2, 3, 4]



The queries are assembled, aligned by index:

Index	queryType	student1	student2
0	Friend	1	2
1	Friend	2	3
2	Total	1	4

Students will start as discrete groups {1}, {2}, {3} and {4}. Students 1 and 2 become friends with the first query, as well as students 2 and 3 in the second. The new groups are {1, 2}, {2, 3} and {4} which simplifies to {1, 2, 3} and {4}. In the third query, the number of friends for student 1 = 3 and student 4 = 1 for a Total = 4. Notice that student 3 is indirectly part of the circle of friends of student 1 because of student 2.

Function Description

Complete the function *getTheGroups* in the editor below. For a query of type *Total* with an index of i , the function must return an array of integers where the value at each index j denotes the answer.

getTheGroups has the following parameter(s):

int n: the number of students

string queryType: an array of queries to be processed

Test Results



sentences for queries[i], the response for that query should be [-1].

Constraints

- $1 \leq n \leq 10^4$
- $1 \leq q \leq 10^4$
- The number of words in any sentence or query is in the range [1-10].
- Each word has at most 11 characters.
- No word appears in more than 10 sentences.
- Each word consists of uppercase and lowercase English alphabetic letters only (i.e., the character class [a-zA-Z]).

Input Format for Custom Testing

Sample Case 0

Sample Input 0

STDIN	Function
3	sentences[] size n = 3
jim likes mary	sentences = ['jim likes mary', 'kate likes tom', 'tom does not like jim']
kate likes tom	
tom does not like jim	
2	queries[] size q = 2
jim tom	queries = ['jim tom', 'likes']
likes	

Sample Output 0

2
0 1

Explanation 0

Perform the following $q = 2$ queries on sentences = ["jim likes mary", "kate likes tom", "tom does not like jim"]:

0. Find the indices of sentences containing both the words "jim" and "tom". The only sentence containing both words is located at index 2, so the array [2] is stored in index 0 of the results array.
1. Find the indices of sentences containing the word "likes". This word appears in sentences[0] and sentences[1], so the array [0, 1] is stored in index 1 of the results array.

Finally return the array [[2], [0, 1]] as the answer

Sample Case 1

Sample Case 2

https://www.hackerrank.com/test/890ps2jl3t1/questions/76f971h31hk

the final answer is $36 - 3 = 33$, and $33 \bmod 1000000007 = 33$.

Function Description

Complete the function `countDiceSequences` in the editor below. The function must return the number of different possible sequences, modulo $(10^9 + 7)$.

`countDiceSequences` has the following parameters:

`N`: The number of times to roll the die.

`rollMax`: An array of 6 integers, each `rollMax[i]` denotes the maximum number of consecutive rolls of number i where $1 \leq i \leq 6$.

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq \text{rollMax}[i] \leq 50$

Input Format For Custom Testing

Sample Case 0

Sample Input For Custom Testing

```
3
6
1
1
1
1
1
1
1
```

Sample Output

150

Explanation

$N = 3$

`rollMax = [1, 1, 1, 1, 1, 1]`

In the first roll, any outcome is possible. In each of the subsequent rolls, there are only 5 possible outcomes because no roll can be repeated more than once. For three rolls, then, there are $6 * 5 * 5 = 150$ possible sequences, and $150 \bmod 1000000007 = 150$.

Sample Case 1

student 7 because of student 2.

https://www.hackerrank.com/test/890ps2jl3t1/questions/a9tiqq6kpm1

Function Description

Complete the function *getTheGroups* in the editor below. For a query of type *Total* with an index of *j*, the function must return an array of integers where the value at each index *j* denotes the answer.

getTheGroups has the following parameter(s):

- int n*: the number of students
- string queryType[q]*: an array of query type strings
- int student1[q]*: an array of student integer ID's
- int student2[q]*: an array of student integer ID's

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq q \leq 10^5$
- $1 \leq \text{students1}[i], \text{students2}[i] \leq n$
- queryTypes[i]* are in the set {'Friend', 'Total'}

Input Format for Custom Testing

Sample Case 0

Sample Input 0

```
STDIN      Function
-----
3          + n = 3
2          + queryType[] size q = 2
Friend     + query = ['Friend', 'Total']
Total
2          + students1[] size q = 2
1          + students1 = [1, 2]
2
2          + students2[] size q = 2
2          + students2 = [2, 3]
3
```

Sample Output 0

3

Explanation 0

Initial	Friend 1 2	Total 2 3
1 2	1 2	1 2

Test Re

https://www.hackerrank.com/test/890ps2jl3t1/questions/a9tiqq6kpm1

- $1 \leq q \leq 10^5$
- $1 \leq \text{students1}[i], \text{students2}[i] \leq n$
- $\text{queryTypes}[i]$ are in the set {'Friend', 'Total'}

Input Format for Custom Testing

Sample Case 0

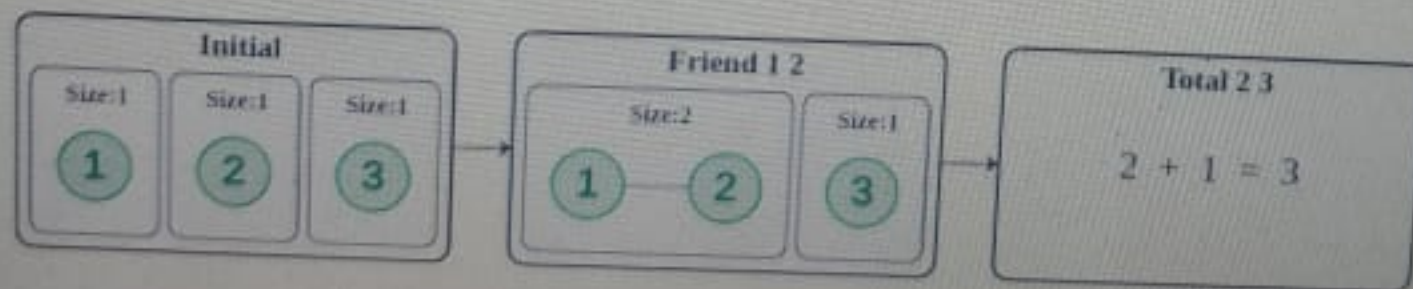
Sample Input 0

STDIN	Function
3	$\rightarrow n = 3$
2	$\rightarrow \text{queryType}[]$ size $q = 2$
Friend	$\rightarrow \text{query} = ['\text{Friend}', '\text{Total}']$
Total	
2	$\rightarrow \text{students1}[]$ size $q = 2$
1	$\rightarrow \text{students1} = [1, 2]$
2	
2	$\rightarrow \text{students2}[]$ size $q = 2$
2	$\rightarrow \text{students2} = [2, 3]$
3	

Sample Output 0

3

Explanation 0



There are $n = 3$ students labeled with the IDs $\{1, 2, 3\}$. Perform the following $q = 2$ queries:

0. Friend 1 2: Students 1 and 2 become friends.
1. Total 2 3: Find the total number of students in the collective friend groups of students 2 and 3. Student 2 is in a group with two friends (i.e., $\{1, 2\}$) and student 3 is in a group with one friend (i.e., $\{3\}$), so store $2 + 1 = 3$ in index 0 of the return array.

After performing all the queries, return the array $[3]$ as the answer.

Sample Case 1

2. Biased Die Simulator

A die simulator selects a random number from 1 to 6 for each roll. The programmer introduced a constraint in the generator: it cannot roll the number i more than $rollMax[i]$ times consecutively, where $1 \leq i \leq 6$. Determine the number of distinct sequences that may be rolled in some number of rolls, N . A sequence is distinct if there is at least one element that differs when compared to all other sequences. The number might be quite large, so return the value modulo 1000000007 , (10^9+7) .

Example:

$N = 2$

$rollMax = [1, 2, 1, 2, 1, 2]$

Looking at $rollMax$, using 1-based indexing, there can be 1 roll of 1, 2 consecutive rolls of 2, 1 roll of 3, 2 consecutive rolls of 4 and so on.

There will be 2 rolls of the die. For the first roll, there are six possible outcomes, 1 through 6. Looking at $rollMax$, there are three values that cannot be rolled in consecutive throws, values 1, 3, 5. If there are no constraints on the die, there are 6 possible outcomes in each roll, so $6 * 6 = 36$ possible combinations. It is known that the sequences (1, 1), (3, 3) and (5, 5) cannot occur, so the final answer is $36 - 3 = 33$, and $33 \bmod 1000000007 = 33$.

Function Description

Complete the function `countDiceSequences` in the editor below. The function must return the number of different possible sequences, modulo $(10^9 + 7)$.

`countDiceSequences` has the following parameters:

N : The number of times to roll the die.

$rollMax$: An array of 6 integers, each $rollMax[i]$ denotes the maximum number of consecutive rolls of number i where $1 \leq i \leq 6$.

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq rollMax[i] \leq 50$

> Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

Language

1 >

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23 > tr

Test Result

1. Simple Text Queries

In this challenge, you will be given an array of *sentences* and an array of *queries*. Determine which sentences contain all of the words of a query. If no sentence contains all of the words, the answer to that query is `[-1]`.

Example

```
sentences = ['bob and alice like to text each other', 'bob does not like to ski but does not like to fall', 'Alice likes to ski']
queries = ['bob alice', 'alice', 'like', 'non occurrence']
```

The results of the *queries* are:

0. `sentences[0] -> [0]`
1. `sentences[0] -> [0]`
2. `sentences[0], sentences[1], sentences[1] -> [0, 1, 1]`
3. none match -> `[-1]`

Return a 2-dimensional integer array: `[[0], [0], [0, 1, 1], [-1]]`.

Note: The word *like* in `queries[2]` does not match *likes* in `sentences[2]`. The word *alice* does not match *Alice* in sentence 2. Matches must be exact.

After each query has been processed, add an array of indices of the matching *sentences* to the answer array. If all of the words of a query occur multiple times in a sentence, the index of that `sentences[i]` should occur that number of times (see result 2 above for an example). The order of the words' occurrence in the sentence does not matter.

Function Description

Complete the function `textQueries` in the editor below.

`textQueries` has the following parameter(s):

`string sentences[n]`: sentence strings that consist of space-separated words

`string queries[q]`: an array strings of space-separated words

Returns

`int[q][i]`: A 2-dimensional array where each element `i` is an array of indices relating to `queries[i]`. If there are no matching sentences for `queries[i]`, the response for that query should be `[-1]`.

Constraints

Language

1 > #in

9

10 /*

11 * C

12 *

13 * T

14 * T

15 *

16 *

17 */

18

19 vecto

queri

20

21 }

22

23 > int ma

Test Results

